

제04장.

- 오라클 접속에서 새로운 유저 생성

```
create user user04 identified by pass;  
grant connect, resource, dba to user04;
```

- 학사관리 기본 함수 작성

1) 학과별 평균급여를 구하여 학과별 평균급여 대비 교수들의 급여 백분율을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC01(i_dept professors.dept%TYPE)  
RETURN NUMBER  
AS  
    v_dept_avg NUMBER :=0;  
BEGIN  
    SELECT AVG(salary) INTO v_dept_avg FROM professors WHERE dept=i_dept;  
    RETURN v_dept_avg;  
END FUNC01;
```

```
select func01('전산') from dual;  
select professors.*, func01(dept) from professors;  
select professors.*, func01(dept), TO_CHAR(salary/func01(dept) * 100, '999.99') from professors;
```

2) 교수 번호를 입력 받아 이 교수가 지도하는 학생 수를 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC02(i_pcode professors.pcode%TYPE)  
RETURN NUMBER  
AS  
    v_count NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_count FROM students WHERE advisor=i_pcode;  
    RETURN v_count;  
END FUNC02;
```

```
select func02('228') from dual;  
select professors.*, func02(pcode) from professors;
```

3) 교수 번호를 입력 받아 이 교수가 담당하는 강좌 수를 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC03(i_pcode professors.pcode%TYPE)  
RETURN VARCHAR2  
AS  
    v_count NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_count FROM courses WHERE instructor=i_pcode;  
    RETURN v_count;  
END FUNC03;
```

```
select func03('509') from dual;  
select professors.*, func03(pcode) from professors;  
select professors.*, func03(pcode) as 담당과목수, func02(pcode) as 지도학생수 from professors;
```

• Pipelined Function

결과를 한꺼번에 모아서 반환하는 일반 함수와 달리 결과가 생성될 때마다 호출자에게 즉시 데이터를 한 줄씩 던져주는 특수한 함수이다. 대량의 데이터를 처리할 때 메모리 부하를 줄이고 첫 번째 행이 출력되는 속도를 획기적으로 개선할 수 있다. 실무에서는 단순 조회가 아니라 복잡한 계산이나 비즈니스 로직을 거쳐 데이터를 가공해서 내보낼 때 매우 유용하다.

1) 교수번호를 입력받아 해당 교수가 지도하는 학생들의 평균을 구하여 목록으로 출력한다.

```
//1) 객체 타입 및 컬렉션 타입(중첩 테이블) 생성
CREATE OR REPLACE TYPE TYPE_STU_OBJ AS OBJECT(
    scode CHAR(8), sname VARCHAR(15), dept VARCHAR(30), savg NUMBER);
/ --PL/SQL 문장을 실행한다, 새로운 줄에 단독으로 입력해야 한다.

CREATE OR REPLACE TYPE TYPE_STU_TBL IS TABLE OF TYPE_STU_OBJ;
/
```

```
--2) PIPELINED 키워드와 PIPE ROW 명령어를 사용하여 Pipelined 함수 작성
CREATE OR REPLACE FUNCTION FUNC04(i_pcode professors.pcode%TYPE)
RETURN TYPE_STU_TBL PIPELINED
AS
    v_avg NUMBER;
BEGIN
    FOR s IN (SELECT * FROM students WHERE advisor=i_pcode) LOOP
        SELECT AVG(grade) INTO v_avg FROM enrollments WHERE scode=s.scode;
        PIPE ROW(TYPE_STU_OBJ(s.scode, s.sname, s.dept, v_avg)); --결과가 나올 때마다 즉시 호출자에게 전달
    END LOOP;
    RETURN; --값을 반환하기 않고 종료로만 사용
END FUNC04;
```

```
--3) 함수를 실행할 때는 일반 SELECT 문에서 TABLE() 함수로 감싸서 사용한다.
select * from TABLE(func04('221'));
```

2) 교수번호를 입력받아 해당 교수가 담당하는 강좌들의 평균을 구하여 목록으로 출력한다.

```
CREATE OR REPLACE TYPE TYPE_COU_OBJ AS OBJECT(
    lcode CHAR(4), lname VARCHAR(200), hours NUMBER, cavg NUMBER);
/

CREATE OR REPLACE TYPE TYPE_COU_TBL AS TABLE OF TYPE_COU_OBJ;
/
```

```
CREATE OR REPLACE FUNCTION FUNC05(i_pcode professors.pcode%TYPE)
RETURN TYPE_COU_TBL PIPELINED
AS
    v_avg NUMBER;
BEGIN
    FOR c IN (SELECT * FROM courses WHERE instructor=i_pcode) LOOP
        SELECT AVG(grade) INTO v_avg FROM enrollments WHERE lcode=c.lcode;
        PIPE ROW(TYPE_COU_OBJ(c.lcode, c.lname, c.hours, v_avg));
    END LOOP;
    RETURN;
END FUNC05;
```

```
select * from TABLE(func05('509'));
```

• REF CURSOR

REF CURSOR는 오라클 PL/SQL에서 쿼리(Query) 결과 세트(Result Set)에 대한 포인터(Pointer) 역할을 하는 데이터 타입이다. 일반 커서(Static Cursor)가 선언 시점에 SQL 문이 고정되는 것과 달리, REF CURSOR는 실행 시점에 원하는 SQL 문을 연결할 수 있는 동적 커서이다. 주로 프로시저에서 조회한 결과를 자바(Java)나 다른 외부 애플리케이션으로 넘겨줄 때 핵심적으로 사용된다.

1) 교수번호를 입력받아 해당 교수가 지도하는 학생 목록을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC06(
    i_pcode professors.pcode%TYPE)
RETURN SYS_REFCURSOR
AS
    cur_stu SYS_REFCURSOR;
BEGIN
    OPEN cur_stu FOR SELECT * FROM students WHERE advisor=i_pcode;
    RETURN cur_stu;
END FUNC06;
```

```
var cur refcursor; -- 1. 커서를 담을 변수 선언
exec :cur := func06('228'); -- 2. 함수 실행
print cur; -- 3. 결과 출력
```

2) 교수번호를 입력받아 해당 교수가 담당하는 강좌 목록을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC07(
    i_pcode professors.pcode%TYPE)
RETURN SYS_REFCURSOR
AS
    cur_cou SYS_REFCURSOR;
BEGIN
    OPEN cur_cou FOR SELECT * FROM courses WHERE instructor=i_pcode;
    RETURN cur_cou;
END FUNC07;
```

```
var cur refcursor;
exec :cur := func07('228');
print cur;
```

3) 학번을 입력 받아 지도 교수 이름을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC08(
    i_scode students.scode%TYPE)
RETURN VARCHAR2
AS
    v_advisor students.advisor%TYPE;
    v_pname professors.pname%TYPE;
BEGIN
    SELECT advisor INTO v_advisor FROM students WHERE scode=i_scode;
    SELECT pname INTO v_pname FROM professors WHERE pcode=v_advisor;
    RETURN v_pname;
END FUNC08;
```

```
select func08('92414029') from dual;
select students.*, func08(scode) from students;
```

4) 학번을 입력 받아 수강 신청한 강좌 수를 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC09(i_scode students.scode%TYPE)
RETURN NUMBER
AS
    v_count NUMBER;
BEGIN
    SELECT count(*) INTO v_count FROM enrollments WHERE scode=i_scode;
    RETURN v_count;
END FUNC09;
```

```
select func09('92414029') as 수강강좌수 from dual;
select students.*, func09(scode) as 수강강좌수 from students;
```

5) 학번을 입력 받아 수강 신청한 강좌들의 평균을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC10(i_scode students.scode%Type)
RETURN NUMBER
AS
    v_avg NUMBER;
BEGIN
    SELECT avg(grade) INTO v_avg FROM enrollments WHERE scode=i_scode;
    RETURN v_avg;
END FUNC10;
```

```
select TO_CHAR(func10('92414029'), '999.99') as 강좌평균 from dual;
select students.*, TO_CHAR(func10(scode), '999.99') as 강좌평균 from students;
select students.*, TO_CHAR(func10(scode), '999.99') as 강좌평균, func09(scode) as 수강강좌수 from students;
select students.*, TO_CHAR(func10(scode), '999.99') as 평균, func09(scode) as 수강강좌수,
    func08(scode) as 지도교수 from students;
```

6) 학생들의 과목평균을 입력 받아 학점을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC11(i_avg NUMBER)
RETURN VARCHAR2
AS
    v_grade VARCHAR2(2);
BEGIN
    IF i_avg >= 90 THEN
        IF i_avg >=95 THEN v_grade := 'A+'; ELSE v_grade := 'A0'; END IF;
    ELSIF i_avg >= 80 THEN
        IF i_avg >=85 THEN v_grade := 'B+'; ELSE v_grade := 'B0'; END IF;
    ELSIF i_avg >= 70 THEN
        IF i_avg >=75 THEN v_grade := 'C+'; ELSE v_grade := 'D0'; END IF;
    ELSIF i_avg >= 60 THEN
        IF i_avg >=65 THEN v_grade := 'D+'; ELSE v_grade := 'D0'; END IF;
    ELSE v_grade := 'F';
    END IF;
    RETURN v_grade;
END FUNC11;
```

```
select students.*, TO_CHAR(func10(scode), '999.99') as 평균,
    func11(func10(scode)) as 평점 from students;
```

7) 학생 테이블에서 학과, 학년별 평균을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC12(
    i_dept students.dept%TYPE,
    i_year students.year%TYPE)
RETURN NUMBER
AS
    v_avg NUMBER;
BEGIN
    SELECT avg(e.grade) INTO v_avg FROM enrollments e, students s
    WHERE e.scode=s.scode GROUP BY s.dept, s.year HAVING dept=i_dept AND year=i_year;
    RETURN v_avg;
END FUNC12;
```

```
select func12('전산', 3) as 학과학년평균 from dual;
select students.*, func12(dept, year) as 학과학년평균 from students;
select students.*, func12(dept, year) as 학과학년평균, func10(scode) as 평균 from students;
```

8) 학생 번호를 입력 받아 해당 학생이 수강 신청한 강좌 정보 목록을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC13(
    i_scode students.scode%TYPE)
RETURN SYS_REFCURSOR
AS
    cur_enroll SYS_REFCURSOR;
BEGIN
    OPEN cur_enroll FOR
        SELECT e.lcode, c.lname, e.edate, e.grade, func11(e.grade) 학점
        FROM enrollments e, courses c
        WHERE e.lcode=c.lcode AND scode=i_scode;
    RETURN cur_enroll;
END FUNC13;
```

```
var cur refcursor;
exec :cur := func13('92414029');
print cur;
```

9) 강좌 번호를 입력 받아 담당 교수 이름을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC14(
    i_lcode courses.lcode%TYPE)
RETURN VARCHAR2
AS
    v_instructor courses.instructor%TYPE;
    v_pname professors.pname%TYPE;
BEGIN
    SELECT instructor INTO v_instructor FROM courses WHERE lcode=i_lcode;
    SELECT pname INTO v_pname FROM professors WHERE pcode=v_instructor;
    RETURN v_pname;
END FUNC14;
```

```
select func14('A109') 담당교수명 from dual;
select courses.*, func14(lcode) 담당교수명 from courses;
```

10) 강좌 번호를 입력 받아 이 강좌의 수강 인원수를 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC15(  
    i_lcode courses.lcode%TYPE  
)  
RETURN NUMBER  
AS  
    v_count NUMBER;  
BEGIN  
    SELECT count(*) INTO v_count FROM enrollments WHERE lcode=i_lcode;  
    RETURN v_count;  
END FUNC15;
```

```
select func15('A109') 수강인원 from dual;  
select courses.*, func15(lcode) from courses;
```

11) 강좌 번호를 입력받아 강좌 평균 점수를 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC16(  
    i_lcode courses.lcode%TYPE  
)  
RETURN NUMBER  
AS  
    v_avg NUMBER;  
BEGIN  
    SELECT avg(grade) INTO v_avg FROM enrollments WHERE lcode=i_lcode;  
    RETURN v_avg;  
END FUNC16;
```

```
select TO_CHAR(func16('A109'), '999.99') as 과목평균 from dual;  
select courses.*, TO_CHAR(func16(lcode), '999.99') as 과목평균 from courses;  
select courses.*, TO_CHAR(func16(lcode), '999.99') as 과목평균, func15(lcode) as 수강과목 from courses;  
select courses.*, TO_CHAR(func16(lcode), '999.99') as 과목평균, func15(lcode) as 수강과목, func14(lcode) as 담당교수 from courses;
```

12) 강좌 번호를 입력 받아 해당 강좌를 수강 신청한 학생 정보 목록을 출력한다.

```
CREATE OR REPLACE FUNCTION FUNC17(  
    i_lcode courses.lcode%TYPE  
)  
RETURN SYS_REFCURSOR  
AS  
    cur_enroll SYS_REFCURSOR;  
BEGIN  
    OPEN cur_enroll FOR  
        SELECT rownum, e.scode, s.sname, e.edate, e.grade, func11(e.grade) 학점  
        FROM enrollments e, students s  
        WHERE e.scode = s.scode AND e.lcode=i_lcode;  
    RETURN cur_enroll;  
END FUNC17;
```

```
var cur refcursor;  
exec :cur := func17('A109');  
print cur;
```

- 학사관리 통계

1) 교수 테이블의 학과별, 직급별 인원수와 평균급여를 구한다.

```
CREATE OR REPLACE TYPE TYPE_STATS1_OBJ AS OBJECT(  
    dept VARCHAR2(100), title VARCHAR2(100), persons NUMBER, avg_salary NUMBER);  
CREATE OR REPLACE TYPE TYPE_STATS1_TBL AS TABLE OF TYPE_STATS1_OBJ;
```

```
CREATE OR REPLACE FUNCTION FUNC18  
RETURN TYPE_STATS1_TBL PIPELINED  
AS  
    TYPE TYPE_DEPT IS TABLE OF professors.dept%TYPE;  
    array_dept TYPE_DEPT;  
    TYPE TYPE_TITLE IS TABLE OF professors.title%TYPE;  
    array_title TYPE_TITLE;  
    v_persons NUMBER;  
    v_avg NUMBER;  
BEGIN  
    SELECT DISTINCT(dept) BULK COLLECT INTO array_dept FROM professors;  
    SELECT DISTINCT(title) BULK COLLECT INTO array_title FROM professors;  
    FOR i IN 1..array_dept.COUNT LOOP  
        FOR j IN 1..array_title.COUNT LOOP  
            SELECT COUNT(*), NVL(AVG(salary),0) INTO v_persons, v_avg FROM professors  
                WHERE dept=array_dept(i) AND title=array_title(j);  
            PIPE ROW(TYPE_STATS1_OBJ(array_dept(i), array_title(j), v_persons, v_avg));  
        END LOOP;  
    END LOOP;  
    RETURN;  
END FUNC18;
```

2) 학생 테이블의 학과별, 학년별 인원수를 구한다.

```
CREATE OR REPLACE TYPE TYPE_STATS2_OBJ AS OBJECT(  
    dept VARCHAR2(20), year NUMBER, persons NUMBER);  
CREATE OR REPLACE TYPE TYPE_STATS2_TBL AS TABLE OF TYPE_STATS2_OBJ;
```

```
CREATE OR REPLACE FUNCTION FUNC19  
RETURN TYPE_STATS2_TBL PIPELINED  
AS  
    TYPE TYPE_DEPT IS TABLE OF students.dept%TYPE;  
    array_dept TYPE_DEPT;  
    TYPE TYPE_YEAR IS TABLE OF students.year%TYPE;  
    array_year TYPE_YEAR := TYPE_YEAR(1, 2, 3, 4);  
    v_persons NUMBER;  
BEGIN  
    SELECT DISTINCT(dept) BULK COLLECT INTO array_dept FROM students;  
    FOR i IN 1..array_dept.COUNT LOOP  
        FOR j IN 1..array_year.COUNT LOOP  
            SELECT COUNT(*) INTO v_persons FROM students WHERE dept=array_dept(i) AND year=array_year(j);  
            PIPE ROW(TYPE_STATS2_OBJ(array_dept(i), array_year(j), v_persons));  
        END LOOP;  
    END LOOP;  
    RETURN;  
END FUNC19;
```

3) 강좌테이블의 강좌별 최대수강인원수, 수강인원수를 구한다.

```
CREATE OR REPLACE TYPE TYPE_STATS3_OBJ AS OBJECT(  
    room CHAR(3), cnt NUMBER, persons NUMBER);  
CREATE OR REPLACE TYPE TYPE_STATS3_TBL AS TABLE OF TYPE_STATS3_OBJ;
```

```
CREATE OR REPLACE FUNCTION FUNC20  
RETURN TYPE_STATS3_TBL PIPELINED  
AS  
    TYPE TYPE_ROOM IS TABLE OF courses.room%TYPE;  
    array_room TYPE_ROOM;  
    v_count NUMBER;  
    v_persons NUMBER;  
    v_total NUMBER;  
BEGIN  
    SELECT DISTINCT(room) BULK COLLECT INTO array_room FROM courses;  
    FOR i IN 1..array_room.COUNT LOOP  
        v_count := 0;  
        v_total := 0;  
        FOR c IN (SELECT * FROM courses WHERE room=array_room(i)) LOOP  
            v_count := v_count + 1;  
            SELECT COUNT(*) INTO v_persons FROM enrollments WHERE lcode=c.lcode;  
            v_total := v_total + v_persons;  
        END LOOP;  
        PIPE ROW(TYPE_STATS3_OBJ(array_room(i), v_count, v_total));  
    END LOOP;  
    RETURN;  
END FUNC20;
```

3) 시작일과 종료일을 입력 받아 일별 수강 신청 인원수를 출력한다.

```
CREATE OR REPLACE TYPE TYPE_STATS4_OBJ AS OBJECT (  
    edate DATE, ecount INT);  
CREATE OR REPLACE TYPE TYPE_STATS4_TBL AS TABLE OF TYPE_STATS4_OBJ;
```

```
CREATE OR REPLACE FUNCTION FUNC21(  
    i_sdate IN DATE,  
    i_edate IN DATE  
)  
RETURN TYPE_STATS4_TBL PIPELINED  
AS  
    v_sdate DATE := TO_DATE(i_sdate, 'YY/MM/DD');  
    v_edate DATE := TO_DATE(i_edate, 'YY/MM/DD');  
    v_persons INT;  
BEGIN  
    WHILE v_sdate <= v_edate LOOP  
        SELECT COUNT(*) INTO v_persons FROM enrollments WHERE edate=TO_CHAR(v_sdate, 'YY/MM/DD');  
        PIPE ROW(TYPE_STATS4_OBJ(v_sdate, v_persons));  
        v_sdate := v_sdate + 1;  
    END LOOP;  
    RETURN;  
END FUNC21;
```

```
select * from TABLE(func21('98/03/02', '98/03/10'));
```